

**MAGE**  
Multi-Agent GraphRAG Estimator (MAGE)  
**Software Requirements Specification (SRS)**  
& Technical Whitepaper

Prepared by: Master of the RAG System Builder  
Project Architect & Manager

April 2, 2026

## **Abstract**

The MAGE system is a production-grade, 100% backend Django REST API that functions as an intelligent, never-fail estimation master for IT software projects.

It ingests all historical project data (estimations, actual hours, requirements, tech stacks, team details) into a hybrid GraphRAG + VectorRAG knowledge base. For any new requirement, it behaves like an experienced Project Manager: asks clarification questions when needed, performs detailed task breakdown, delivers precise hour estimates with assumptions/risks/buffers, maintains full conversation context, and continuously learns from user corrections.

This document serves as the single source of truth for developers (implementation guide) and clients/stakeholders (system understanding and value proposition).

# Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>2</b>  |
| 1.1      | Purpose . . . . .                                       | 2         |
| 1.2      | Scope . . . . .                                         | 2         |
| 1.3      | Definitions and Acronyms . . . . .                      | 3         |
| <b>2</b> | <b>Overall Description</b>                              | <b>4</b>  |
| 2.1      | Product Perspective . . . . .                           | 4         |
| 2.2      | Product Functions . . . . .                             | 4         |
| 2.3      | User Classes and Characteristics . . . . .              | 4         |
| <b>3</b> | <b>System Architecture</b>                              | <b>5</b>  |
| 3.1      | High-Level Architecture . . . . .                       | 5         |
| 3.2      | Key Technologies . . . . .                              | 5         |
| <b>4</b> | <b>Detailed Functional Requirements</b>                 | <b>7</b>  |
| 4.1      | Authentication (FR-01) . . . . .                        | 7         |
| 4.2      | Ingestion (FR-02) . . . . .                             | 7         |
| 4.3      | Estimation Workflow (FR-03 & FR-04) . . . . .           | 7         |
| 4.4      | Feedback Loop (FR-05) . . . . .                         | 7         |
| <b>5</b> | <b>Data Design</b>                                      | <b>8</b>  |
| 5.1      | PostgreSQL Models . . . . .                             | 8         |
| 5.2      | Neo4j Graph Schema . . . . .                            | 8         |
| <b>6</b> | <b>Development &amp; Implementation Guidelines</b>      | <b>9</b>  |
| <b>7</b> | <b>Deployment &amp; Operations</b>                      | <b>10</b> |
| <b>A</b> | <b>Appendix A: Jira Task Table (Buffered 248 Hours)</b> | <b>11</b> |
| <b>B</b> | <b>Appendix B: Sample API Endpoints</b>                 | <b>12</b> |

# List of Figures

|     |                                          |   |
|-----|------------------------------------------|---|
| 3.1 | High-Level System Architecture . . . . . | 5 |
|-----|------------------------------------------|---|

# Chapter 1

## Introduction

### 1.1 Purpose

The purpose of the MAGE system is to automate and dramatically improve the accuracy of IT project effort estimation by leveraging your organization's past project data through advanced Retrieval-Augmented Generation (RAG) techniques, specifically **GraphRAG** combined with a **multi-agent workflow**.

Traditional estimation methods and simple RAG systems suffer from incomplete context, hallucinations, and lack of organizational memory. MAGE overcomes these limitations through:

- Hybrid knowledge storage (Neo4j Graph + pgvector)
- Stateful multi-agent collaboration orchestrated by LangGraph
- Self-correction mechanisms (Self-RAG + CRAG)
- Continuous learning via instant feedback loops
- End-to-end JWT authentication and production readiness

### 1.2 Scope

#### In Scope (Phase 1):

- Complete backend-only REST API built with Django 5.1 and Django REST Framework
- Ingestion pipeline for historical project documents (PDF, Excel, JSON, CSV)
- Six specialized AI agents working together for estimation
- Stateful conversation management with clarification and correction support
- Full observability and monitoring
- Docker-based deployment

#### Out of Scope (Future Phases):

- Web or mobile frontend UI
- Advanced analytics dashboard
- Multi-language support
- Integration with external project management tools (Jira, etc.)

## 1.3 Definitions and Acronyms

- **GraphRAG**: Retrieval using knowledge graph relationships for multi-hop reasoning.
- **LangGraph**: Library for building controllable, stateful multi-agent applications.
- **Self-RAG / CRAG**: Techniques where agents critique and improve their own retrieval and reasoning.
- **Thread**: A stateful estimation conversation identified by a unique ID.
- **MAGE**: Multi-Agent GraphRAG Estimator (this system).

# Chapter 2

## Overall Description

### 2.1 Product Perspective

MAGE transforms static historical project data into a living, intelligent estimation engine. It combines the strengths of:

- Vector similarity search (semantic understanding)
- Graph traversal (relationship-aware reasoning)
- Agentic workflows (collaborative problem solving)
- Human-in-the-loop feedback (continuous improvement)

The result is a system that delivers estimations with citations to similar past projects, clear task breakdowns, risk analysis, and exact hour ranges — while learning from every correction you make.

### 2.2 Product Functions

1. **Data Ingestion** — Process past project artifacts and build the hybrid knowledge base.
2. **New Estimation** — Accept requirements, handle multi-turn clarification, and return detailed estimates.
3. **Conversation Management** — Maintain context across multiple interactions within a thread.
4. **Feedback & Learning** — Incorporate user corrections and actual hours into the graph for future improvements.
5. **History & Search** — Retrieve past estimations and find similar projects quickly.

### 2.3 User Classes and Characteristics

- **Project Managers / Estimators** (Primary users): Submit requirements and review outputs.
- **Technical Leads / Architects**: Provide detailed feedback on technical breakdowns.
- **Administrators**: Manage data ingestion and monitor system health via APIs.

# Chapter 3

## System Architecture

### 3.1 High-Level Architecture

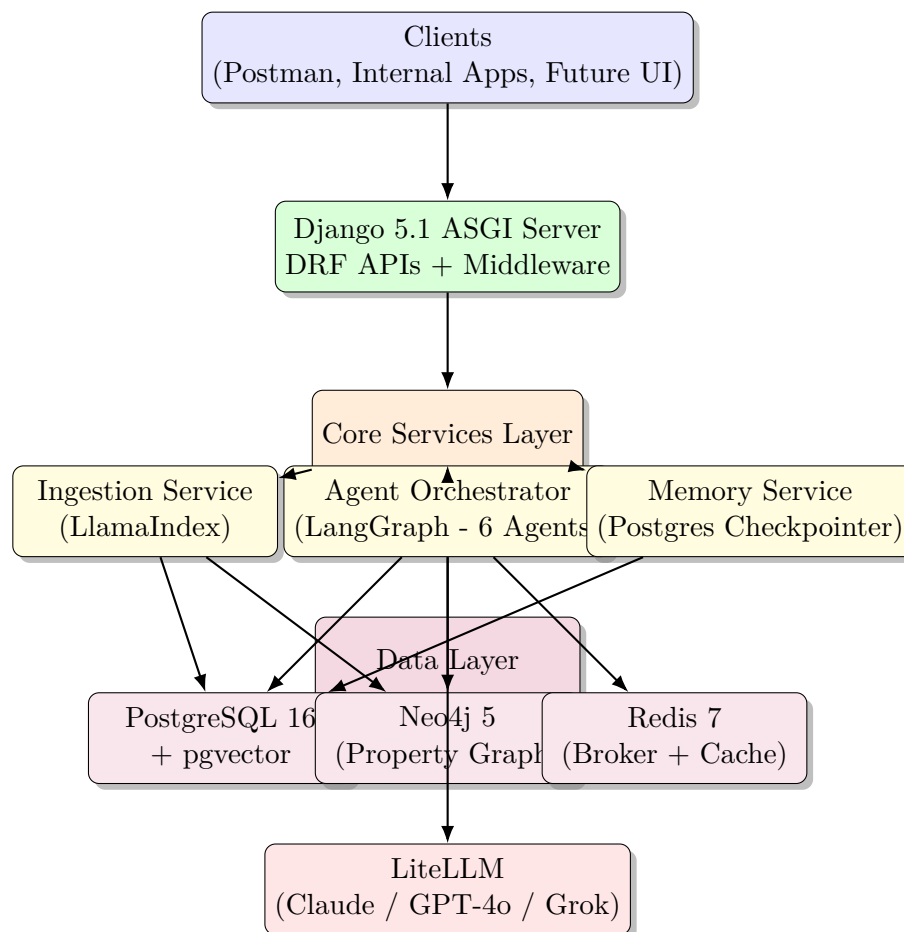


Figure 3.1: High-Level System Architecture

### 3.2 Key Technologies

- Framework: Django 5.1 + Django REST Framework
- Orchestration: LangGraph 0.2 (stateful agents)
- Indexing: LlamaIndex 0.12 (PropertyGraphIndex)

- Databases: PostgreSQL 16 with pgvector, Neo4j 5
- Queue: Celery + Redis
- LLM Abstraction: LiteLLM
- Authentication: SimpleJWT
- Observability: LangSmith, Prometheus, Grafana, Sentry



## Chapter 4

# Detailed Functional Requirements

### 4.1 Authentication (FR-01)

All APIs must be protected by JWT. The token must be propagated to Celery tasks and every LangGraph agent node for user-scoped data access and audit logging.

### 4.2 Ingestion (FR-02)

Support multiple file formats. The pipeline must extract entities and relationships to populate both the graph and vector store.

### 4.3 Estimation Workflow (FR-03 & FR-04)

The system shall execute the following agent sequence (orchestrated by LangGraph):

1. Clarifier Agent — Detect ambiguity and generate clarification questions
2. Retriever Agent — Perform hybrid search + GraphRAG + reranking
3. Decomposer Agent — Break requirements into hierarchical tasks/subtasks
4. Estimator Agent — Calculate hours using historical analogies and rules
5. Validator Agent — Self-critique using Self-RAG/CRAG
6. Feedback Agent — Handle corrections and update knowledge graph

Final output must include:

- Detailed task breakdown
- Hour estimates (min / expected / max) per task and total
- Assumptions, risks, and contingency buffer
- Citations to similar past projects

### 4.4 Feedback Loop (FR-05)

User corrections and actual hours must immediately update the Neo4j graph so that future estimations become more accurate.

# Chapter 5

## Data Design

### 5.1 PostgreSQL Models

Key Django models include:

- Project, Requirement, TaskBreakdown
- TechStack, TeamRole
- EstimationThread, FeedbackLog
- Vector fields for embeddings

### 5.2 Neo4j Graph Schema

Nodes: Project, Task, TechStack, Estimate, LessonsLearned Key Relationships: SIMILAR\_TO, USES\_TECH, TOOK\_HOURS, HAD\_RISK, LEARNED\_FROM

## Chapter 6

# Development & Implementation Guidelines

Developers must follow the buffered Jira task list (248 hours total) in strict sequential order, with special attention to:

- Completing authentication before any agent development
- Documenting DB schema before implementing models
- Testing each agent node independently with mocked LLM
- Ensuring Celery tasks never block the main Django process
- Using LangSmith traces for every estimation run

## Chapter 7

# Deployment & Operations

The system is designed for containerized deployment using Docker. Production recommendations include Kubernetes or managed platforms (Railway/Render) with separate services for Django, Celery workers, databases, and monitoring.

## Appendix A

# Appendix A: Jira Task Table (Buffered 248 Hours)

(Refer to the detailed task table provided in previous communications — includes all tasks from EPIC-0 to EPIC-9 with extra buffer hours.)

## Appendix B

# Appendix B: Sample API Endpoints

- `POST /api/estimate/` — Start new estimation
- `GET /api/estimate/{thread_id}/status/` — Check status
- `POST /api/estimate/{thread_id}/respond/` — Reply to clarification
- `POST /api/estimate/{thread_id}/feedback/` — Submit corrections
- `POST /api/ingest/projects/` — Ingest historical data